

Formation Engine Methodology

XYZ File Formats

VSEPR-SIM Atomistic Engine

Formats: `.xyz` `.xyza` `.xyzc` `.xyzf` `.xyzfull`

Edition 4 — Expanded Reference with Optional INFO Encoding

VSEPR-SIM Project

Version 4.2.0 — April 24, 2026

Abstract

All VSEPR-SIM molecular I/O uses plain-text, line-oriented files. Four formats share a common structural grammar; each adds fields to the preceding one. This document is the fourth-edition reference for the `.xyz`, `.xyza`, `.xyzc`, `.xyzf`, and `.xyzfull` file formats, including grammar specifications, field semantics, coordinate conventions, checkpoint and trajectory usage, bounding-box encoding, implementation mapping, typical observable ranges, the rich replay container format, the optional `INFO` tail block, compute kernel tags, and the new `INFO_ENCODED` optional encoding mechanism introduced in Edition 4 for guaranteed backward compatibility with legacy readers.

Contents

What Is New in Edition 4 — v4.2.0

Edition 4 adds optional encoding of the INFO tail block. All prior formats and their semantics are unchanged and fully backward-compatible with Edition 3 readers.

New in this edition:

- **§B — Optional INFO Block Encoding (INFO_ENCODED):** An alternative to the plain INFO tail block. Block content may be encoded as **plain**, **base64**, **hex**, or **zlib+base64**. Legacy readers that halt after atom data lines are unaffected.
- **Encoding schemes:** **plain** (identity), **base64** (RFC 4648), **hex** (lower-case), **zlib+base64** (DEFLATE then base64).
- **Writer config:** `encode_info_block` and `info_encoding` added to `XYZWriterConfig`.
- **Decoder API:** `InfoBlockDecoder::decode` / `InfoBlockDecoder::encode` in namespace `vsepr::io`.
- **New source files:** `include/io/info_block_codec.hpp` and `src/io/info_block_codec.cpp`.
- **New error codes:** `DUAL_INFO_BLOCK`, `UNKNOWN_ENCODING`, `INFO_DECODE_ERROR`.
- **Feature matrix and INFO applicability table** updated.
- **Per-format backward-compatibility** guarantees stated explicitly.

Rule: a file must carry either `INFO` or `INFO_ENCODED`, never both. `INFO_ENCODED` plain is valid but redundant; prefer plain `INFO`.

What Was New in Edition 3 — v4.1.0 (retained for reference)

Edition 3 added `.xyzfull` (the rich replay and render container) and the optional plain `INFO` tail block for all formats. All prior formats (`.xyz`, `.xyza`, `.xyzc`, `.xyzf`) remained unchanged and fully backward-compatible with Edition 2 readers.

.xyzfull — At-a-Glance Spec Card**File Structure**

```

XYZFULL
version 1
units angstrom fs kcal/mol
mode <string>
energy_layers <N> <names...>
static_count <S>
dynamic_count <D>
properties ...
STATIC
<id> <type> <sym> x y z q cg_id
FRAMES
FRAME <k> time <t>
<dynamic rows>
END_FRAME
INFO or INFO_ENCODED <scheme>
<content / encoded blob>
TOTAL_FRAMES <N>
DT <dt>
END_INFO or END_INFO_ENCODED

```

Energy Layers

```

e0  U_total
e1  U_bond
e2  U_vdw
e3  U_coul
e4  U_ext
e5  U_thermal

```

Count: $k \in \{4, 5, 6\}$. Declare in header.
Do not invent arbitrary counts.

Key Constraints

- File *must* start with XYZFULL
- TOTAL_FRAMES and DT required (in decoded INFO)
- Parsed frame count = TOTAL_FRAMES
- Dynamic IDs unique within frame
- Static IDs $\neq$ dynamic IDs
- $\|\mathbf{o}\| > 10^{-12}$ (orientation vector)
- Energy layer count: 4, 5, or 6 only
- Unknown `step_tag`: warn, not fatal
- INFO and INFO_ENCODED: mutually exclusive

bead	sphere / dot
rod	capsule / line
plate	rectangle proxy
gear	low-poly wheel
pipe	tube segment
crystal	lattice proxy

Not CAD. Visual truth only.

NOT a checkpoint

.xyzfull $\neq$ restartable state.
Use .xyzc to restart MD.
Use XYZFullStreamWriter for long runs.

§1 Overview

The VSEPR-SIM atomistic engine reads and writes molecular geometry, dynamics, and checkpoint data using four progressively richer plain-text formats. Each format extends the previous one by appending per-atom or per-frame metadata while preserving backward compatibility with simple line-oriented parsers.

Table 1: Format hierarchy.

Extension	Name	Use case
<code>.xyz</code>	Standard XYZ	Static geometry snapshot
<code>.xyza</code>	Extended XYZ	Charges, velocities, forces
<code>.xyzc</code>	Checkpoint XYZ	Restartable MD state
<code>.xyzf</code>	Formation XYZ	Multi-frame trajectory
<code>.xyzfull</code>	Full Replay XYZ	Rich replay/render container

1.1 Unit Conventions

All formats share a single, non-negotiable unit system:

Table 2: Unit system (all formats).

Quantity	Unit	SI equivalent
Coordinates	Å (angstrom)	10^{-10} m
Energy	kcal/mol	4.184×10^3 J/mol
Velocity	Å/fs	10^5 m/s
Force	kcal/(mol Å)	6.947×10^{13} N/mol
Charge	elementary e	1.602×10^{-19} C
Time	fs	10^{-15} s
Temperature	K	kelvin

1.2 Coordinate System

Right-handed Cartesian (x, y, z) in Å. Origin is arbitrary; the conventional choice is near the centre of mass. Coordinate precision: six decimal places ($\Rightarrow$ resolution ≈ 0.1 pm).

§2 Standard XYZ (`.xyz`)

2.1 Grammar

```
<N>
<comment>
<sym> <x> <y> <z>
... (N lines)
```

- **Line 1:** atom count N (integer ≥ 1).
- **Line 2:** free-text comment (may be empty).
- **Lines 3–($N+2$):** element symbol + three Cartesian coordinates, whitespace-separated.

2.2 Comment Line Convention

The comment line follows a structured-but-optional pattern:

```
<name> | E = <val> kcal/mol | <notes>
```

Fields are pipe-delimited. The energy key `E =` is recognised by the reader for energy-tracking contexts.

2.3 Example: Water

```
3
H2O | E = -6.245 kcal/mol
O   0.000000  0.000000  0.119262
H   0.000000  0.763239 -0.477049
H   0.000000 -0.763239 -0.477049
```

2.4 Parsing Notes

- Element symbols are case-sensitive: `He` is helium, `HE` is rejected.
- Trailing whitespace on atom lines is ignored.
- Blank lines between the comment and the first atom line are *not* permitted.
- Coordinate fields must parse as IEEE 754 double-precision values.

§3 Extended XYZ (.xyza)

Adds per-atom scalar and vector properties after the (x, y, z) fields.

3.1 Grammar

```
<N>
<comment> properties=<list>
<sym> <x> <y> <z> [<q> <vx> <vy> <vz> <fx> <fy> <fz> <e>]
```

The `properties` key declares which optional columns are present. Column order is fixed; omission is permitted only from the right.

3.2 Property Columns

3.3 Properties Declaration Syntax

```
properties="charge:velocity"
properties="charge:velocity:force:energy"
```

Unrecognised keys trigger a parse warning but do not abort the read.

Table 3: Extended property columns (fixed order).

Key	Symbol	Unit	Columns
charge	q_i	e	1
velocity	$\mathbf{v}_i$	$\text{\AA}/\text{fs}$	3
force	$\mathbf{F}_i$	$\text{kcal}/(\text{mol } \text{\AA})$	3
energy	ε_i	kcal/mol	1

3.4 Example: H₂O with Charges and Velocities

```

3
Water MD frame 1 properties="charge:velocity"
O  0.00  0.00  0.12 -0.834  0.00  0.00  0.00
H  0.00  0.76 -0.48  0.417  0.12  0.08 -0.03
H  0.00 -0.76 -0.48  0.417 -0.12  0.08 -0.03

```

3.5 Charge Semantics

The charge field q_i represents the effective partial charge used in Coulomb interactions. It is *not* an oxidation state.

§4 Checkpoint XYZ (.xyzc)

Extends .xyza with a simulation-state header block required to restart an interrupted MD run without loss of trajectory continuity.

4.1 Header Block

```

CHECKPOINT
step      <integer>
time      <float> fs
dt        <float> fs
T_target  <float> K
seed      <integer>
box       <ax> <ay> <az>
END_CHECKPOINT
<N>
<comment>
<atom lines with charge + velocity + force>

```

4.2 Header Field Semantics

4.3 Restart Protocol

1. Header parsed; integrator state restored.
2. Atom block read as a standard .xyza frame.

Table 4: Checkpoint header fields.

Field	Type	Meaning
<code>step</code>	integer	MD step number at checkpoint
<code>time</code>	float	Accumulated simulation time (fs)
<code>dt</code>	float	Integrator timestep (fs)
<code>T_target</code>	float	Thermostat set-point (K); 0 = NVE ensemble
<code>seed</code>	integer	RNG seed for reproducible continuation
<code>box</code>	3 floats	Box side lengths (Å); omit for open boundary

3. Neighbour lists reconstructed; never serialised into checkpoint.
4. If `box` line present, PBC activated.

4.4 Checkpoint vs. Trajectory

A `.xyzc` file contains exactly one frame. For multi-frame output, use `.xyzf` (§??).

§5 Formation XYZ (`.xyzf`)

A multi-frame format: one or more `.xyz` or `.xyza` blocks concatenated in a single file.

5.1 Grammar

```
<N>          <- frame 1
<comment>
<atom lines>
<N>          <- frame 2
<comment>
<atom lines>
...
```

Each frame is self-contained and independently parseable.

5.2 Comment Line Convention (Multi-Frame)

```
Step 0 | E = -12.30 kcal/mol | FIRE iter
Step 1 | E = -13.11 kcal/mol | FIRE iter
```

5.3 Frame Independence

- Atom count N may vary between frames.
- The `properties` declaration may differ between frames.
- Readers must re-parse the header of each frame.

5.4 Typical Uses

Table 5: Common .xyzf applications.

Application	Comment line pattern
FIRE minimisation trace	Step <k> E = <val> kcal/mol FIRE iter
NVT trajectory dump	Step <k> E = <val> kcal/mol T = <T> K
Reaction path (NEB)	Image <k> E = <val> kcal/mol dE = <dE>
Conformer scan	Conf <k> E = <val> kcal/mol <dihedral>

§6 Bounding Box and Periodic Boundary Conditions

When PBC are active, box geometry is stored in the .xyzc header (§??) or as comment-line keys in any format.

6.1 Comment-Line Encoding

```
Lattice="ax 0 0 0 ay 0 0 0 az" pbc="T T T"
```

- Box vectors in Å, flattened 3×3 row-major matrix.
- pbc flags: T = periodic, F = open.
- Compatible with ASE Extended XYZ convention.

6.2 Box Vector Interpretation

$$\mathbf{L} = \begin{pmatrix} a_x & 0 & 0 \\ 0 & a_y & 0 \\ 0 & 0 & a_z \end{pmatrix} \quad (1)$$

The cell volume is $V = \det(\mathbf{L}) = a_x \cdot a_y \cdot a_z$.

6.3 Minimum Image Convention

$$\Delta r_\alpha = r_\alpha^{(j)} - r_\alpha^{(i)} - L_\alpha \cdot \text{round}\left(\frac{r_\alpha^{(j)} - r_\alpha^{(i)}}{L_\alpha}\right), \quad \alpha \in \{x, y, z\} \quad (2)$$

§7 Bond Auto-Detection

Bond connectivity is inferred from geometry, never stored in the file.

$$d_{ij} < s \cdot (r_{\text{cov}}(Z_i) + r_{\text{cov}}(Z_j)) \implies \text{bond}(i, j) \quad (3)$$

where s is the scale factor (default $s = 1.2$).

7.1 Design Rationale

Bonds are derived data, not ground truth. Stale bond tables are a silent-corruption vector. Caches never become truth.

§8 Implementation Mapping

Table 6: Source files implementing the XYZ format family.

File	Role
include/io/xyz_format.hpp	Public API: XYZReader, XYZWriter, XYZAReader, XYZAWriter
src/io/xyz_format.cpp	Reader / writer implementation
include/thermal/xyzc_format.hpp	Checkpoint reader/writer declarations
src/thermal/xyzc_format.cpp	Checkpoint reader/writer implementation
include/io/xyzfull_format.hpp	Replay container: XYZFullReader, XYZFullWriter, XYZFullStreamWriter
src/io/xyzfull_format.cpp	Full replay reader / writer / streaming writer
include/render/xyzfull_replay.hpp	Low-poly replay renderer declarations
src/render/xyzfull_replay.cpp	Replay renderer and CG proxy reconstruction
include/io/info_block_codec.hpp	InfoBlockDecoder / InfoBlockEncoder (Edition 4)
src/io/info_block_codec.cpp	Encoding/decoding: base64, hex, zlib+base64

All reader and writer types reside in namespace `vsepr::io`.

8.1 Reader API Pattern

```
// Standard / Extended / Checkpoint / Formation
auto frame = vsepr::io::XYZReader::read(molecule.xyz);
auto frame = vsepr::io::XYZAReader::read(dynamics.xyza);
auto state = vsepr::io::XYZCReader::read(restart.xyzc);
auto traj = vsepr::io::XYZFReader::read_all(trajjectory.xyzf);
// Full replay container
auto replay = vsepr::io::XYZFullReader::read(replay.xyzfull);
// replay.info_encoding tells you the scheme used
```

8.2 Writer API Pattern

```
vsepr::io::XYZWriter::write(output.xyz, atoms, comment);
vsepr::io::XYZAWriter::write(output.xyza, atoms, props, comment);
vsepr::io::XYZCWriter::write(checkpoint.xyzc, state);

vsepr::io::XYZFWriter writer(trajjectory.xyzf, /*append=*/true);
writer.write_frame(atoms, comment);
```

```
// Full replay (streaming)
vsepr::io::XYZFullStreamWriter sw(replay.xyzfull);
sw.write_header(header);
sw.write_static_block(statics);
sw.begin_frames();
for (const auto& frame : frames) sw.write_frame(frame);
sw.write_footer(info, total_frames, dt);

// Edition 4: encoded INFO block
vsepr::io::XYZWriterConfig cfg;
cfg.encode_info_block = true;
cfg.info_encoding      = "base64"; // or "hex", "zlib+base64"
vsepr::io::XYZFWriter writer2(trajjectory.xyzf, cfg);
```

§9 Typical Energy and Observable Ranges

Table 7: Typical observable magnitudes for sanity checking.

Observable	Typical range	Unit
Total bonded energy	-10^2 to -10^5	kcal/mol
Relative conformer energy	0.1 – 10	kcal/mol
Activation barrier	5 – 50	kcal/mol
Force (unrelaxed atom)	1 – 100	kcal/(mol Å)
Force convergence criterion	< 0.01	kcal/(mol Å)
Velocity (300 K, C atom)	~ 0.02	Å/fs
Partial charge (O in water)	−0.82 to −0.84	e
Partial charge (H in water)	+0.41 to +0.42	e

Values outside these bounds in production output should trigger audit warnings.

§10 Integration with the Identity–State Framework

The XYZ formats serve as the serialisation layer for the particle identity vector $\mathbf{I}_i$ defined in §0 of the Formation Engine methodology.

Table 8: Identity vector components and their file-format encoding.

Component	Symbol	File encoding
Nuclear identity	Z_i	Element symbol (looked up to Z)
Mass identity	A_i	Implicit via Z_i
Charge participation	Q_i	charge column in .xyza
Structural role	Σ_i	Not serialised; inferred at load
Stability class	Λ_i	Not serialised; computed post-load
Provenance memory	Θ_i	Comment-line metadata

$\mathcal{K}_{\text{geom}}$ maps to the lattice matrix from the `Lattice` key or `box` header. $\mathcal{K}_{\text{comp}}$ is reconstructed at load from element composition and geometry.

§11 Format Comparison Matrix

Table 9: Feature comparison across formats.

Feature	.xyz	.xyza	.xyzc	.xyzf	.xyzfull
Atom count + coordinates	✓	✓	✓	✓	✓
Partial charges		✓	✓	opt.	static
Velocities		✓	✓	opt.	✓
Forces		✓	✓	opt.	
Per-atom energy		✓	✓	opt.	layered
Integrator state			✓		
Thermostat / RNG			✓		
Box geometry (header)			✓		
Box geometry (comment key)	✓	✓	✓	✓	
Multi-frame				✓	✓
Static / dynamic separation					✓
CG orientation vector					✓
Energy layer decomposition					✓
STEP proxy tag					✓
Low-poly render support					✓
Optional plain <code>INFO</code> block	opt.	opt.	opt.	opt.	struct.
Optional encoded <code>INFO</code> block	opt.	opt.	opt.	opt.	opt.

§12 Error Handling and Validation

Readers enforce these validation rules at parse time:

1. **Atom count mismatch.** $\Rightarrow$ `XYZParseError::INCOMPLETE_FRAME`.
2. **Unknown element.** $\Rightarrow$ `XYZParseError::UNKNOWN_ELEMENT`.
3. **Malformed coordinates.** $\Rightarrow$ `XYZParseError::BAD_COORDINATE`.
4. **Column count mismatch.** Warn and zero-fill (fail-soft for velocities/forces; fail-hard for coordinates).
5. **Checkpoint integrity.** $\Rightarrow$ `XYZCParseError::NO_HEADER`.
6. **Dual `INFO` blocks.** Both `INFO` and `INFO_ENCODED` in same file $\Rightarrow$ `XYZParseError::DUAL_INFO_BLOCK`.
7. **Unknown encoding scheme.** $\Rightarrow$ `XYZParseError::UNKNOWN_ENCODING`.
8. **Decoding failure.** $\Rightarrow$ `XYZParseError::INFO_DECODE_ERROR`.

§13 Full Replay XYZ (.xyzfull)

The `.xyzfull` format is the terminal form of the XYZ family: a rich replay and render container that captures bead/CG dynamics, orientation state, energy layers, and STEP proxy tags in a single streamable file.

It is not a checkpoint. It is not a canonical simulation state. It is a replay artifact. Restarting physics from .xyzfull data is not supported and would be unsound.

13.1 Design Rationale

- Separates *static* particles (walls, fixtures) from *dynamic* particles (beads, CG bodies). Statics stored once; dynamic state per frame.
- Orientation vectors (**o**) enable low-poly proxy reconstruction.
- Energy layers declared in header, stored inline, supporting 4–6 named components.
- STEP proxy tags link each bead to a part identifier for renderer-side look-up.

13.2 Grammar

```

XYZFULL
version 1
units angstrom fs kcal/mol
mode <mode_string>
energy_layers <N> <name0> ... <nameN-1>
static_count <S>
dynamic_count <D>
properties id type sym x y z vx vy vz ox oy oz cg_id step_tag e0 [e1 ...]

STATIC
<id> <type> <sym> <x> <y> <z> <q> <cg_id>
... (S lines)

FRAMES
FRAME <k> time <t>
<id> <type> <sym> <x> <y> <z> <vx> <vy> <vz> <ox> <oy> <oz> <cg_id> <
  step_tag> <e0> [...]
... (D lines)
END_FRAME
...

INFO                                <- plain (Edition 3+)
  or
INFO_ENCODED <scheme>              <- encoded (Edition 4+)
<content>
TOTAL_FRAMES <N>
DT <dt>
END_INFO / END_INFO_ENCODED

```

13.3 File-Level Rules

- File must begin with XYZFULL.
- `version` must be a supported integer (currently 1).
- `energy_layers` declares count $k \in \{4, 5, 6\}$ followed by names.

- Must contain exactly one of `INFO` or `INFO_ENCODED`.
- `TOTAL_FRAMES` and `DT` must be present in decoded `INFO` content.
- Parsed frame count must equal `TOTAL_FRAMES`.

13.4 Energy Layer Convention

Table 10: Recommended 6-layer energy decomposition for `.xyzfull`.

Index	Key	Description
e0	<code>U_total</code>	Total potential energy
e1	<code>U_bond</code>	Bonded interactions
e2	<code>U_vdw</code>	van der Waals
e3	<code>U_coul</code>	Coulomb/electrostatic
e4	<code>U_ext</code>	External field
e5	<code>U_thermal</code>	Thermal / thermostat contribution

A reduced 4-layer mode (`e0=U_total`, `e1=U_internal`, `e2=U_external`, `e3=U_state`) is also valid. Do not create arbitrary layer counts.

13.5 Orientation Vector and STEP Proxy

$$\|\mathbf{o}\| = \sqrt{o_x^2 + o_y^2 + o_z^2} > 10^{-12} \quad (4)$$

Violation produces a warning; the frame is not discarded.

Table 11: CG proxy shape map.

Bead type	Proxy geometry
bead	Sphere / dot
rod	Capsule / line
plate	Rectangle proxy
gear	Low-poly wheel
pipe	Tube segment
crystal	Lattice proxy

13.6 Minimal Example (plain INFO)

```

XYZFULL
version 1
units angstrom fs kcal/mol
mode bead_cg_replay
energy_layers 6 U_total U_bond U_vdw U_coul U_ext U_thermal
static_count 2
dynamic_count 2
properties id type sym x y z vx vy vz ox oy oz cg_id step_tag e0 e1 e2 e3
            e4 e5

```

```

STATIC
900 wall X 0.000000 0.000000 0.000000 0.0 -1
901 wall X 10.00000 0.000000 0.000000 0.0 -1

FRAMES
FRAME 0 time 0.000000
1 bead C 0.000000 0.000000 0.000000 0.00 0.00 0.00 1.00 0.00 0.00 10
    STEP_A -12.3 -1.2 -4.1 -2.0 -0.4 -0.2
2 bead O 1.000000 0.000000 0.000000 0.00 0.01 0.00 0.00 1.00 0.00 10
    STEP_A -10.1 -1.0 -3.8 -1.7 -0.2 -0.1
END_FRAME
FRAME 1 time 0.010000
1 bead C 0.010000 0.000000 0.000000 0.01 0.00 0.00 1.00 0.00 0.00 10
    STEP_B -12.0 -1.1 -4.0 -1.9 -0.4 -0.2
2 bead O 1.010000 0.000100 0.000000 0.01 0.01 0.00 0.00 1.00 0.00 10
    STEP_B -9.9 -1.0 -3.7 -1.6 -0.2 -0.1
END_FRAME

INFO
Replay stores dynamic bead positions and CG orientation vectors.
Static particles are stored once and reconstructed during playback.
TOTAL_FRAMES 2
DT 0.010000

```

13.7 Data Model

```

struct XYZFullHeader {
    int version = 1;
    std::string units;
    std::string mode;
    std::vector<std::string> energy_layers;
    int static_count = 0;
    int dynamic_count = 0;
};

struct XYZFullStatic {
    int id; std::string type; std::string sym;
    double x, y, z; double q; int cg_id;
};

struct XYZFullDynamic {
    int id; std::string type; std::string sym;
    double x, y, z;
    double vx, vy, vz;
    double ox, oy, oz; // orientation vector
    int cg_id; std::string step_tag;
    std::array<double, 6> energy;
};

```

```
struct XYZFullFrame {
    int frame_index; double time;
    std::vector<XYZFullDynamic> dynamic;
};

struct XYZFullData {
    XYZFullHeader header;
    std::vector<XYZFullStatic> static_particles;
    std::vector<XYZFullFrame> frames;
    std::string info_block; // decoded plain text
    std::string info_encoding; // scheme used (Edition 4)
    int total_frames;
    double dt;
};
```

13.8 Reader / Writer API

```
namespace vsepr::io {

class XYZFullReader {
public:
    static XYZFullData read(const std::string& path);
};

class XYZFullWriter {
public:
    static void write(const std::string& path, const XYZFullData& data);
};

class XYZFullStreamWriter {
public:
    explicit XYZFullStreamWriter(const std::string& path);
    void write_header(const XYZFullHeader& header);
    void write_static_block(const std::vector<XYZFullStatic>& statics);
    void begin_frames();
    void write_frame(const XYZFullFrame& frame);
    void write_footer(const std::string& info, int total_frames, double dt
    );
};

} // namespace vsepr::io
```

13.9 Replay Flow

```
void replay_xyzfull(const XYZFullData& data) {
    RenderWorld world;
    world.load_static(data.static_particles);
    for (const auto& frame : data.frames) {
        world.update_dynamic(frame.dynamic);
    }
}
```

```

        world.update_cg_orientation(frame.dynamic);
        render_low_poly(world);
        sleep_fs(data.dt);
    }
}

```

13.10 Shell Commands

```

export xyzfull last      out.xyzfull
export xyzfull history replay.xyzfull
replay xyzfull           replay.xyzfull
tui      xyzfull         replay.xyzfull
convert xyzf xyzfull     trajectory.xyzf replay.xyzfull

# Edition 4 encoded-INFO options
export xyzfull history replay.xyzfull --encode-info base64
export xyzfull history replay.xyzfull --encode-info zlib+base64

```

13.11 Validation Rules

1. File must begin with XYZFULL.
2. `version` must be a supported value.
3. Energy layer count must be 4, 5, or 6.
4. Exactly one of `INFO` or `INFO_ENCODED` must be present.
5. `TOTAL_FRAMES` and `DT` must appear in decoded INFO content.
6. Parsed frame count must equal `TOTAL_FRAMES`.
7. Dynamic IDs must be unique within each frame.
8. Static IDs must not collide with dynamic IDs.
9. $\|\mathbf{o}\| > 10^{-12}$; violation is warned, not fatal.
10. Unknown `step_tag` is permitted with a warning.

§14 Optional INFO Block and Compute Kernel Tags

Any format in the XYZ family may carry an optional INFO tail block. In `.xyzfull` this block is structural (the file is invalid without `TOTAL_FRAMES` and `DT`). For the other formats, the block is *optional*: readers must not fail if it is absent. See §?? for the optional encoded form (`INFO_ENCODED`).

14.1 INFO Block Grammar (All Formats)

```

INFO
<free text lines>
KERNEL <kernel_id> <backend> <threads> [<flags>]
KERNEL ...
TOTAL_FRAMES <N>      # .xyzf and .xyzfull only
DT <dt>                # .xyzfull only; optional annotation elsewhere
END_INFO              # optional terminator; assumed at EOF

```


14.2 KERNEL Directive

Table 12: `KERNEL` directive fields.

Field	Type	Description
<code>kernel_id</code>	string	Logical kernel name (e.g. <code>nbody_vdw</code>)
<code>backend</code>	string	<code>cpu</code> , <code>cuda</code> , <code>opencl</code> , <code>simd</code> , <code>tui</code>
<code>threads</code>	integer	Thread / warp count at time of write
<code>flags</code>	string*	Space-separated optional flags

14.2.1 Standard Kernel IDs

Table 13: Standard kernel identifiers.

Kernel ID	Description
<code>nbody_vdw</code>	Pairwise van der Waals (Lennard-Jones or similar)
<code>nbody_coul</code>	Pairwise Coulomb / electrostatics
<code>bond_harmonic</code>	Harmonic bond stretching
<code>angle_cosine</code>	Cosine angle bending
<code>torsion_fourier</code>	Fourier torsion series
<code>thermostat_nhc</code>	Nosé–Hoover chain thermostat
<code>integrator_vv</code>	Velocity-Verlet integrator step
<code>cg_orient</code>	CG orientation propagation (bead replay)
<code>render_proxy</code>	Low-poly proxy reconstruction (<code>.xyzfull</code> only)

14.2.2 Optional Flags

- `pb` — periodic boundary conditions active
- `cutoff=<r>` — non-bonded cutoff radius (Å)
- `skin=<r>` — neighbour-list skin distance (Å)
- `dt=<val>` — timestep used by this kernel (fs)
- `reduced` — energy written in reduced 4-layer mode
- `stream` — file produced by `XYZFullStreamWriter`

14.3 INFO Block per Format

14.4 Example: `.xyzf` with INFO and Kernel Tags

```

3
Step 0 | E = -12.30 kcal/mol | FIRE iter
C 0.000000 0.000000 0.000000
O 1.230000 0.000000 0.000000
H 2.100000 0.800000 0.000000
3
Step 1 | E = -13.11 kcal/mol | FIRE iter

```

Table 14: INFO block applicability across formats.

Field	.xyz	.xyza	.xyzc	.xyzf	.xyzfull
Free-text notes	opt.	opt.	opt.	opt.	opt.
KERNEL lines	opt.	opt.	opt.	opt.	opt.
TOTAL_FRAMES	—	—	—	opt.	req.
DT	—	—	—	opt.	req.
END_INFO	opt.	opt.	opt.	opt.	opt.
Encoded form allowed	yes	yes	yes	yes	yes

```

C 0.001000 0.000000 0.000000
O 1.229000 0.000000 0.000000
H 2.099000 0.799000 0.000000

INFO
FIRE minimisation trace, 2 steps.
KERNEL  bond_harmonic   cpu  4    dt=0.010000
KERNEL  nbond_vdw        cpu  4    cutoff=12.0 skin=2.0
KERNEL  integrator_vv    cpu  4    dt=0.010000
TOTAL_FRAMES 2
END_INFO

```

14.5 Reader Behaviour

1. No INFO token: silently succeed; KERNEL list is empty.
2. Unknown KERNEL IDs: stored as-is, warned in verbose mode.
3. Unrecognised flags: stored as raw strings; no parse error.
4. For .xyzfull: missing TOTAL_FRAMES or DT is a hard error.
5. If INFO_ENCODED present and reader does not implement §??: silently skip block; must not emit a fatal error.

§15 Optional INFO Block Encoding

Edition 4 introduces an alternative to the plain INFO tail block: the INFO_ENCODED block. Its purpose is to guarantee that legacy tools which scan or post-process XYZ files cannot accidentally corrupt or misinterpret structured metadata embedded at the end of the file.

Backward-compatibility guarantee. Readers that pre-date Edition 4 and halt after the atom data lines (correct behaviour for .xyz, .xyza, .xyzc, .xyzf) will never encounter INFO_ENCODED and are entirely unaffected. For .xyzfull, the token INFO_ENCODED will be encountered; conformant readers must treat an unrecognised block-start token as a non-fatal skip condition.

15.1 Grammar

```
INFO_ENCODED <scheme>
<encoded blob line 1>
<encoded blob line 2>
...
END_INFO_ENCODED
```

- <scheme> is a single whitespace-free token (§??).
- The encoded blob may span any number of lines. Line breaks are insignificant and stripped before decoding. Use 76-character line wrapping for readability.
- END_INFO_ENCODED is the mandatory block terminator. EOF is *not* a valid substitute.
- The decoded content is treated identically to a plain INFO block body, including all KERNEL directives and (for .xyzfull) the required TOTAL_FRAMES and DT fields.

15.2 Encoding Schemes

Table 15: Supported INFO_ENCODED encoding schemes.

Token	Standard	Notes
plain	identity	Same bytes as plain INFO; valid but redundant.
base64	RFC 4648 §4	Default recommended encoding. Line-wrapped at 76 characters.
hex	lowercase nibbles	Two hex digits per byte; space-free.
zlib+base64	RFC 1950 + RFC 4648	DEFLATE then base64. Use for INFO blocks exceeding 4 KiB.

No other scheme tokens are defined. Writers must not invent private scheme names. Readers must emit `XYZParseError::UNKNOWN_ENCODING` for any unrecognised token.

15.3 Backward Compatibility per Format

15.4 Writer Configuration

```
struct XYZWriterConfig {
    // ... existing fields from Edition 3 ...

    bool        encode_info_block = false;    // default: emit plain INFO
    std::string info_encoding      = "base64"; // scheme when encoding is
    on
};
```

When `encode_info_block` is `false` (default), writers emit a plain INFO block as in Edition 3. This flag has no effect if no INFO content is present.

Table 16: Impact of `INFO_ENCODED` on legacy readers by format.

Format	Legacy reader behaviour
<code>.xyz</code>	Stops after N atom lines. <code>INFO_ENCODED</code> never reached. Zero impact.
<code>.xyza</code>	Same as <code>.xyz</code> . Zero impact.
<code>.xyzc</code>	Stops after atom block following <code>END_CHECKPOINT</code> . Zero impact.
<code>.xyzf</code>	Stops after last frame's atom lines. Zero impact.
<code>.xyzfull</code>	Reader encounters <code>INFO_ENCODED</code> after final <code>END_FRAME</code> . A conformant Edition-3 reader treats unknown block tokens as skip-to-EOF; a non-conformant reader may fail. Prefer plain <code>INFO</code> when targeting Edition-3 <code>.xyzfull</code> consumers.

15.5 Decoder / Encoder API

```
namespace vsepr::io {

struct InfoBlockDecoder {
    // Decode encoded blob. Throws XYZParseError on unknown scheme or
    // corrupt blob.
    static std::string decode(const std::string& scheme,
                             const std::string& blob);

    // Encode plain INFO body. Returns blob for INFO_ENCODED /
    // END_INFO_ENCODED.
    static std::string encode(const std::string& scheme,
                             const std::string& plain);

    // Returns true if scheme token is recognised.
    static bool known_scheme(const std::string& scheme) noexcept;
};

} // namespace vsepr::io
```

15.6 Reader Algorithm

1. Scan remaining lines for token `INFO` or `INFO_ENCODED`.
2. **If** `INFO`: parse as plain `INFO` block (Edition 3 behaviour).
3. **If** `INFO_ENCODED`:
 - (a) Read scheme token from the same line.
 - (b) Collect lines until `END_INFO_ENCODED` (mandatory terminator).
 - (c) Strip inter-line whitespace from the blob.
 - (d) Call `InfoBlockDecoder::decode(scheme, blob)`.
 - (e) Parse decoded string as plain `INFO` block body.
4. **Neither token**: silently succeed; `INFO` is empty.
5. **Both tokens**: emit `XYZParseError::DUAL_INFO_BLOCK`.

15.7 Example: .xyzf with Base64-Encoded INFO

The decoded content is identical to the plain example in §??.

```
3
Step 0 | E = -12.30 kcal/mol | FIRE iter
C 0.000000 0.000000 0.000000
O 1.230000 0.000000 0.000000
H 2.100000 0.800000 0.000000
3
Step 1 | E = -13.11 kcal/mol | FIRE iter
C 0.001000 0.000000 0.000000
O 1.229000 0.000000 0.000000
H 2.099000 0.799000 0.000000
INFO_ENCODED base64
RklSRSBtaW5pbWlzYXRpb24gdHJhY2UsIDIgc3RlcHMuCktFUk5FTCAgYm9u
ZF9oYXJtb25pYyAgIGNwdSAgIDQgICBkdD0wLjAxMDAwMApLRVJORUwgIG5i
b2R5X3ZkdYAgICAgICBjcHUgICA0ICAgY3V0b2ZmPTEyLjAgc2tpbj0yLjAK
SOVStkVMICBpbmRlZ3JhdG9yX3Z2ICAgY3B1ICAgNCAgIGROPTAuMDEwMDAw
C1RPVEFMX0ZSQU1FUyAyCg==
END_INFO_ENCODED
```

15.8 Example: .xyzfull with Base64-Encoded INFO

```
XYZFULL
version 1
units angstrom fs kcal/mol
mode bead_cg_replay
energy_layers 4 U_total U_internal U_external U_state
static_count 1
dynamic_count 1
properties id type sym x y z vx vy vz ox oy oz cg_id step_tag e0 e1 e2 e3

STATIC
900 wall X 0.000000 0.000000 0.000000 0.0 -1

FRAMES
FRAME 0 time 0.000000
1 bead C 0.000000 0.000000 0.000000 0.00 0.00 0.00 1.00 0.00 0.00 10
  STEP_A -12.3 -1.2 -4.1 -2.0
END_FRAME

INFO_ENCODED base64
UmVwbGF5IHNoY3JlcyBkeW5hbWljIGJlYWQgcG9zaXRpb25zLgplRVJORUwg
IGNnX29yaWVudCAgICAgICBjcHUgICA0ICAgY3V0b2ZmPTEyLjAgc2tpbj0yLjAK
TCAgcmVuZGVyX3Byb3h5ICAgIHR1aSAgICAgICAgc3RyZWZtC1RPVEFMX0ZS
QU1FUyAxChRUIDEuMDEwMDAwCg==
END_INFO_ENCODED
```

15.9 Compressed Encoding: zlib+base64

When the INFO block is large, zlib+base64 provides significant size reduction.

1. Encode the plain INFO body as UTF-8 bytes.
2. Apply RFC 1950 zlib DEFLATE compression (level 6 recommended).
3. Encode the compressed bytes as RFC 4648 base64, wrapped at 76 characters.

```
std::string plain = "FIRE_\trace...\nKERNEL_\...\nTOTAL_FRAMES_\2\n";
std::string blob = InfoBlockDecoder::encode("zlib+base64", plain);
// blob is ready between INFO_ENCODED / END_INFO_ENCODED
```

15.10 Shell Command Extensions (Edition 4)

```
# Write with encoded INFO block
export xyzf      out.xyzf      --encode-info base64
export xyzfull  out.xyzfull  --encode-info zlib+base64

# Inspect / decode INFO block from any file
info decode     file.xyzf
info decode     replay.xyzfull

# Re-encode existing plain INFO block
info encode     file.xyzf  --scheme base64  --out file_enc.xyzf
```

§16 Cross-References

Table 17: Related documentation.

Document	Relevance
docs/FILE_FORMATS.md	Prose description of all I/O formats
section0_identity_state.pdf	Formal particle/cell/world definitions
section5_integration.tex	MD timestep and integrator details
ALPHA_MODEL_BOOKLET.tex	Alpha model (Drude charge coupling)
section_fuzzy_ball_model.tex	Coarse-grained bead visualisation
include/io/xyzfull_format.hpp	.xyzfull reader/writer/stream API
include/render/xyzfull_replay.hpp	Low-poly CG replay renderer
include/io/info_block_codec.hpp	Edition 4 INFO encoding/decoding API
docs/xyz_file_formats_r3.tex	Previous revision (Edition 3, preserved)